# Event Sourcing + CQRS em Java – Sistema Bancário

**Documentação de requisitos, regras de implementação e guia completo Versão:** 1.0 | **Stack de referência:** Java 21, Spring Boot 3.x, PostgreSQL 16, Apache Kafka (ou RabbitMQ), Testcontainers

> A implementação de referência é feita à mão (sem framework de ES) para que o padrão fique explícito. Na seção 14 há a comparação com Axon Framework e EventStoreDB, caso prefira não construir o Event Store.

---

## 1. Visão Geral

### 1.1 Objetivo

Construir um núcleo bancário (contas, depósitos, saques, transferências) em que **o estado nunca é sobrescrito**: toda mudança é gravada como um **evento imutável** em um log append-only (Event Sourcing). Leituras são servidas por **modelos de leitura** separados (CQRS), alimentados pelos eventos.

### 1.2 Por que Event Sourcing em banco

| Necessidade bancária | Como o padrão atende |
|---|---|
| Auditoria e conformidade (BACEN, SOX, LGPD) | O histórico completo é o próprio armazenamento, não um log auxiliar. |
| Rastreabilidade de saldo | Saldo = soma dos eventos. Qualquer saldo pode ser reconstruído e explicado. |
| Investigação de fraude e disputas | Time travel: reconstruir o estado de uma conta em qualquer instante. |
| Novos relatórios no futuro | Basta criar uma nova projeção e reprocessar o histórico. |
| Alta concorrência de escrita | Inserções append-only, sem `UPDATE` e sem contenção em linhas de saldo. |

### 1.3 CRUD x Event Sourcing

| Aspecto | CRUD | Event Sourcing |
|---|---|---|
| Persistência | `UPDATE contas SET saldo = 150` | `INSERT` de `ValorDepositado` |
| O que é guardado | Estado atual | Todas as transições de negócio |
| Histórico | Perdido ou em tabela de log | Nativo e completo |
| Concorrência | Lock de linha | Optimistic Concurrency por versão do stream |
| Consulta | Direta | Via projeções (CQRS) |

### 1.4 Quando NÃO usar

Evite ES em CRUDs simples, sem requisito de histórico, ou se a equipe não tem disposição para lidar com consistência eventual e evolução de eventos. Aplique apenas nos agregados em que o histórico agrega valor (Conta, Transferência); cadastros auxiliares podem continuar CRUD.

---

## 2. Escopo

**Dentro do escopo:** abertura e encerramento de conta, depósito, saque, bloqueio/desbloqueio, transferência entre contas (via Saga), extrato e saldo (read models), snapshots, projeções reconstruíveis, idempotência, versionamento de eventos, observabilidade, proteção de dados pessoais.

**Fora do escopo:** autenticação real, integração com PIX/SPB/cartões, KYC, câmbio, multi-região.

---

## 3. Requisitos

### 3.1 Requisitos Funcionais

| ID | Requisito |
|---|---|
| RF-01 | Abrir conta com titular, tipo e limite inicial; gera `ContaAberta`. |

| ID | Requisito |
|---|---|
| RF-02 | Depositar valor positivo em conta ativa; gera `ValorDepositado`. |
| RF-03 | Sacar valor respeitando saldo + limite; gera `ValorSacado` ou rejeita o comando. |
| RF-04 | Bloquear/desbloquear conta; operações de débito são rejeitadas quando bloqueada. |
| RF-05 | Encerrar conta somente com saldo zero; gera `ContaEncerrada`. Depois disso, nenhum comando é aceito. |
| RF-06 | Transferir entre contas de forma consistente, com compensação em caso de falha (Saga). |
| RF-07 | Consultar saldo atual, extrato paginado por período e detalhes da conta (lado de leitura). |
| RF-08 | Reconstruir o estado de uma conta em uma data/versão passada (*time travel*). |
| RF-09 | Todo comando de escrita aceita `Idempotency-Key`; repetições não duplicam efeitos. |
| RF-10 | Projeções podem ser descartadas e reconstruídas a partir do Event Store sem perda. |
| RF-11 | Todo evento carrega metadados: `eventId`, `streamId`, `version`, `occurredAt`, `correlationId`, `causationId`, `userId`. |
| RF-12 | Estornos e correções são feitos por **novos eventos compensatórios**, nunca editando eventos antigos. |

## 3.2 Requisitos Não Funcionais

| ID | Requisito | Meta |
|---|---|---|
| RNF-01 | **Imutabilidade**: eventos não podem sofrer `UPDATE` ou `DELETE` (imposto por permissão e trigger). | 100% |
| RNF-02 | **Consistência forte no write side**: dois comandos concorrentes na mesma conta nunca geram estado inválido. | 100% |

| ID | Requisito | Meta |
|---|---|---|
| RNF-03 | **Latência de comando** p95 (incluindo reidratação com snapshot). | < 100 ms |
| RNF-04 | **Atraso das projeções** (event lag) p95. | < 1 s |
| RNF-05 | **Throughput** de escrita em ambiente de teste. | ⩾ 1.000 eventos/s |
| RNF-06 | **Reconstrução** de uma conta com 10.000 eventos usando snapshot. | < 50 ms |
| RNF-07 | **Durabilidade**: confirmação ao cliente só após commit do evento ( `synchronous_commit = on` ). | Obrigatório |
| RNF-08 | **Segurança/LGPD**: dados pessoais em eventos protegidos por criptografia por titular (*crypto-shredding*). | Obrigatório |
| RNF-09 | **Observabilidade**: logs JSON, métricas, tracing com `correlationId` . | Obrigatório |
| RNF-10 | **Evolução**: eventos antigos continuam legíveis por no mínimo 10 anos (upcasting). | Obrigatório |

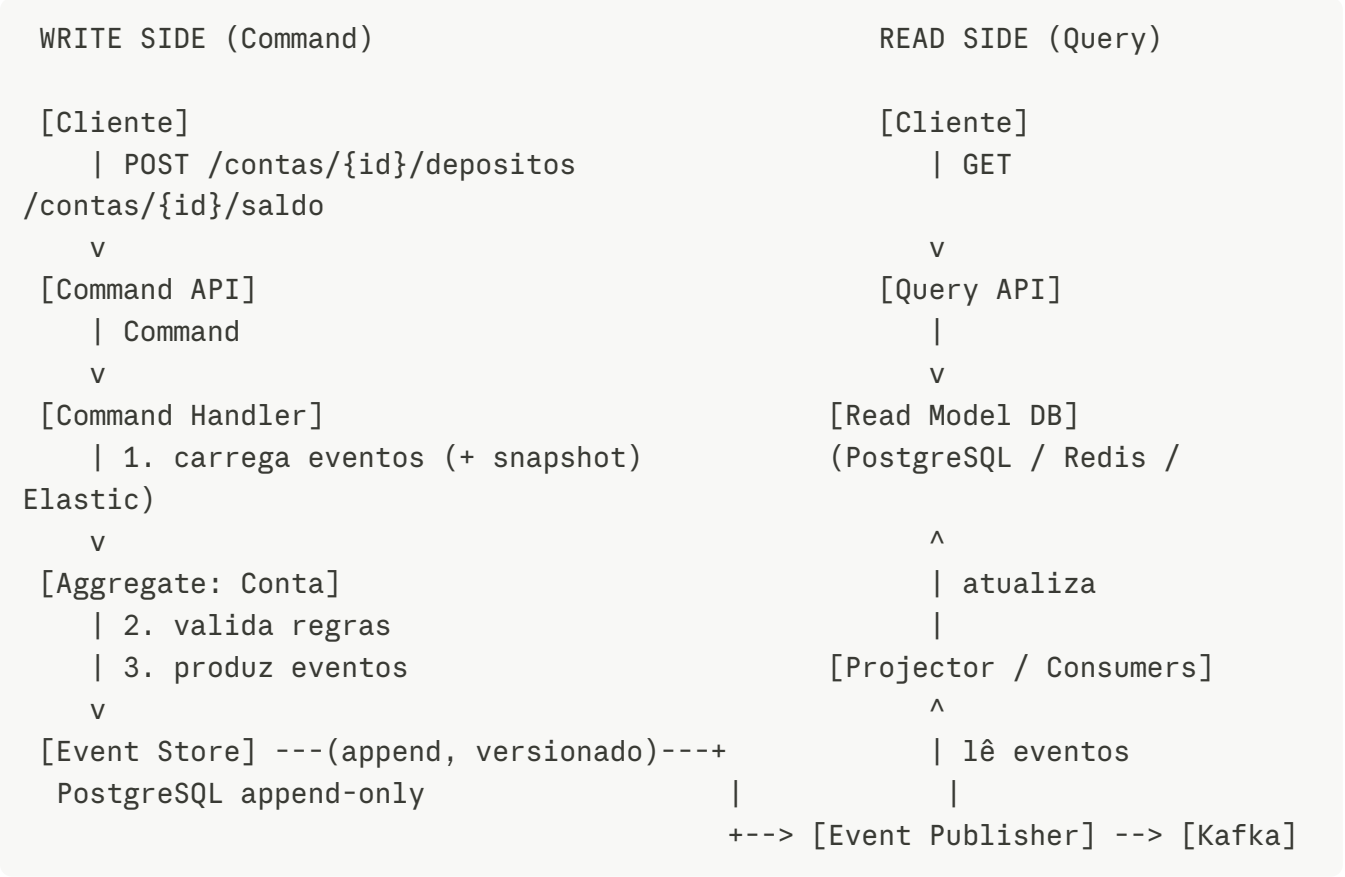
## 3.3 Regras de Negócio

- **RN-01** Valores em `BigDecimal` com escala 2 (ou `long` em centavos). Nunca `double` .
- **RN-02** Depósito e saque exigem valor > 0.
- **RN-03** Saldo disponível = saldo + limite de crédito. Saque acima disso é rejeitado.
- **RN-04** Conta bloqueada não aceita débito (crédito pode ser permitido por política).
- **RN-05** Conta encerrada não aceita nenhum comando.
- **RN-06** Transferência para a mesma conta é proibida.
- **RN-07** Limite diário de saque/transferência configurável por conta.

---

# 4. Arquitetura

## 4.1 Visão geral




## 4.2 Conceitos

| Conceito | Definição no projeto |
|---|---|
| **Command** | Intenção no imperativo ( `DepositarValor` ). Pode ser rejeitada. |
| **Event** | Fato no passado ( `ValorDepositado` ). Imutável, nunca removido. |
| **Aggregate** | `Conta` : fronteira de consistência. Valida regras e produz eventos. |
| **Stream** | Sequência de eventos de um agregado ( `conta-{uuid}` ), versionada 1..N. |
| **Event Store** | Armazenamento append-only dos streams. |
| **Snapshot** | Estado consolidado a cada N eventos, para acelerar a reidratação. |

| Conceito | Definição no projeto |
|---|---|
| **Projection / Read Model** | Visão derivada dos eventos, otimizada para consulta. |
| **Saga / Process Manager** | Coordena fluxo entre agregados (transferência). |

## 4.3 Estrutura de módulos (hexagonal)

```
banco-es/
├── domain/            (agregados, eventos, regras: Java puro, sem Spring)
├── application/       (command handlers, sagas, portas)
├── infrastructure/
|   ├── eventstore/    (JDBC, serialização, snapshots, upcasters)
|   ├── messaging/     (publisher Kafka)
|   └── projections/   (projetores e repositórios de leitura)
├── api-command/       (REST de escrita)
├── api-query/         (REST de leitura)
└── tests/             (unit, integration, property-based)
```

**Regra:** o módulo `domain` não depende de Spring, JDBC, Jackson nem Kafka.

## 4.4 Fluxo de um comando (depósito)

1. API recebe `POST /contas/{id}/depositos` com `Idempotency-Key`.
2. Handler verifica a chave de idempotência.
3. Carrega o stream: último snapshot + eventos posteriores.
4. Reidrata o agregado `Conta` aplicando os eventos.
5. Executa `conta.depositar(valor)`, que valida e retorna `ValorDepositado`.
6. Event Store faz `append` com `expectedVersion`. Se outro comando gravou antes, ocorre `ConcurrencyException` (retry ou 409).
7. Responde `202/201` ao cliente.
8. Publisher entrega o evento ao Kafka; projetores atualizam os read models.

---

# 5. Event Store (PostgreSQL)

## 5.1 Esquema

```
CREATE TABLE events (
    global_position  BIGSERIAL PRIMARY KEY,          -- ordem global para
projeções
    event_id         UUID        NOT NULL UNIQUE,
    stream_id        VARCHAR(100) NOT NULL,          -- 'conta-<uuid>'
    stream_type      VARCHAR(50)  NOT NULL,          -- 'Conta'
    version          INT          NOT NULL,          -- 1..N dentro do
stream
    event_type       VARCHAR(150) NOT NULL,          -- 'ValorDepositado'
    schema_version   INT          NOT NULL DEFAULT 1, -- para upcasting
    payload          JSONB        NOT NULL,
    metadata         JSONB        NOT NULL,          -- correlationId,
causationId, userId
    occurred_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT uq_stream_version UNIQUE (stream_id, version)
);

CREATE INDEX ix_events_stream ON events (stream_id, version);

CREATE TABLE snapshots (
    stream_id   VARCHAR(100) PRIMARY KEY,
    version     INT          NOT NULL,
    state       JSONB        NOT NULL,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE projection_checkpoints (
    projection_name  VARCHAR(100) PRIMARY KEY,
    last_position    BIGINT NOT NULL DEFAULT 0,
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE idempotency_keys (
    key         VARCHAR(100) PRIMARY KEY,
    result      JSONB,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

## 5.2 Imutabilidade imposta pelo banco

```
CREATE OR REPLACE FUNCTION prevent_event_mutation() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'A tabela events é append-only';
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_events_immutable
BEFORE UPDATE OR DELETE ON events
FOR EACH ROW EXECUTE FUNCTION prevent_event_mutation();

-- A aplicação usa um usuário com apenas INSERT e SELECT
REVOKE UPDATE, DELETE ON events FROM app_user;
```

## 5.3 Concorrência otimista

A constraint `UNIQUE (stream_id, version)` é a garantia final. O append é feito com a versão esperada; se outra transação já gravou aquela versão, o banco rejeita e a aplicação lança `ConcurrencyException`.

```
public void append(String streamId, int expectedVersion, List<DomainEvent>
events,
                   EventMetadata meta) {
    try {
        int version = expectedVersion;
        for (DomainEvent e : events) {
            version++;
            jdbc.update(INSERT_SQL, e.eventId(), streamId, "Conta", version,
                        e.getClass().getSimpleName(), e.schemaVersion(),
                        serializer.toJson(e), serializer.toJson(meta));
        }
    } catch (DuplicateKeyException ex) {
        throw new ConcurrencyException(streamId, expectedVersion);
    }
}
```

O `append` roda em uma única transação: ou todos os eventos do comando entram, ou nenhum.

## 5.4 Cuidado com a ordem global

`BIGSERIAL` não garante que a ordem de *commit* seja igual à ordem do número. Uma transação pode obter a posição 10, outra a 11, e a 11 fazer commit antes. Um projetor que lê até 11 e salva o checkpoint pode **pular o evento 10**. Soluções:

- Ler apenas eventos com `xmin` anterior ao menor snapshot de transação ativa (`pg_snapshot_xmin(pg_current_snapshot())`).
- Ou usar um atraso de segurança (ex.: ignorar eventos com menos de 500 ms) e revisitar "buracos" na sequência.
- Ou delegar a ordem global a um broker/EventStoreDB.

Projeções por stream são seguras, pois a ordem dentro do stream é garantida pela versão.

---

# 6. Domínio (Java)

## 6.1 Eventos

```
public sealed interface ContaEvent extends DomainEvent {

    record ContaAberta(UUID eventId, UUID contaId, String titularRef,
                       BigDecimal limite, Instant em) implements ContaEvent
{}

    record ValorDepositado(UUID eventId, UUID contaId, BigDecimal valor,
                           String origem, Instant em) implements ContaEvent
{}

    record ValorSacado(UUID eventId, UUID contaId, BigDecimal valor,
                       Instant em) implements ContaEvent {}

    record ContaBloqueada(UUID eventId, UUID contaId, String motivo,
                          Instant em) implements ContaEvent {}

    record ContaDesbloqueada(UUID eventId, UUID contaId, Instant em)
implements ContaEvent {}

    record ContaEncerrada(UUID eventId, UUID contaId, Instant em) implements
ContaEvent {}
}
```

Observação: `titularRef` é uma referência (id) e não o nome/CPF. Dados pessoais ficam fora do evento ou criptografados (ver seção 11).

## 6.2 Agregado `Conta`

O agregado separa **decisão** (valida e produz eventos) de **mutação** (aplica o evento ao estado, sem regra e sem exceção).

```
public class Conta {

    private UUID id;
    private BigDecimal saldo = BigDecimal.ZERO;
    private BigDecimal limite = BigDecimal.ZERO;
    private Status status = Status.NOVA;
    private int versao = 0;                       // versão do stream
    private final List<ContaEvent> pendentes = new ArrayList<>();

    enum Status { NOVA, ATIVA, BLOQUEADA, ENCERRADA }

    // ---------- Decisão (comandos) ----------
    public static Conta abrir(UUID id, String titularRef, BigDecimal limite,
Clock clock) {
        Conta c = new Conta();
        c.registrar(new ContaAberta(UUID.randomUUID(), id, titularRef,
limite, clock.instant()));
        return c;
    }

    public void depositar(BigDecimal valor, String origem, Clock clock) {
        garantirOperavel();
        exigirPositivo(valor);
        registrar(new ValorDepositado(UUID.randomUUID(), id, valor, origem,
clock.instant()));
    }

    public void sacar(BigDecimal valor, Clock clock) {
        garantirOperavel();
        if (status == Status.BLOQUEADA) throw new
ContaBloqueadaException(id);
        exigirPositivo(valor);
        if (saldo.add(limite).compareTo(valor) < 0) throw new
SaldoInsuficienteException(id);
        registrar(new ValorSacado(UUID.randomUUID(), id, valor,
clock.instant()));
    }

    public void encerrar(Clock clock) {
        garantirOperavel();
        if (saldo.signum() != 0) throw new SaldoNaoZeradoException(id);
        registrar(new ContaEncerrada(UUID.randomUUID(), id,
clock.instant()));
    }

    // ---------- Mutação (apenas aplica fatos, sem validação) ----------
```

```
    private void aplicar(ContaEvent e) {
        switch (e) {
            case ContaAberta ev      -> { id = ev.contaId(); limite =
ev.limite(); status = Status.ATIVA; }
            case ValorDepositado ev  -> saldo = saldo.add(ev.valor());
            case ValorSacado ev      -> saldo = saldo.subtract(ev.valor());
            case ContaBloqueada ev   -> status = Status.BLOQUEADA;
            case ContaDesbloqueada ev-> status = Status.ATIVA;
            case ContaEncerrada ev   -> status = Status.ENCERRADA;
        }
        versao++;
    }

    private void registrar(ContaEvent e) { aplicar(e); pendentes.add(e); }

    public static Conta reconstruir(List<ContaEvent> historico) {
        Conta c = new Conta();
        historico.forEach(c::aplicar);   // não adiciona em 'pendentes'
        return c;
    }

    private void garantirOperavel() {
        if (status == Status.NOVA || status == Status.ENCERRADA) throw new
ContaInvalidaException(id);
    }
    private void exigirPositivo(BigDecimal v) {
        if (v == null || v.signum() <= 0) throw new
ValorInvalidoException(v);
    }

    public List<ContaEvent> eventosPendentes() { return
List.copyOf(pendentes); }
    public int versaoPersistida() { return versao - pendentes.size(); }
}
```

## 6.3 Regras de ouro do domínio

1. O método `aplicar` **nunca** valida nem lança exceção: o evento já aconteceu.
2. O método `aplicar` é determinístico: sem `Instant.now()`, `UUID.randomUUID()`, I/O ou chamadas externas. O tempo e os ids entram na decisão (via `Clock`) e ficam gravados no evento.
3. Um comando afeta **um único agregado** por transação.
4. Eventos são `record`s imutáveis, nomeados no passado.

5. Validações de negócio ficam no agregado; validação de formato (nulo, tamanho) fica na borda da API.

---

# 7. Lado de Comando (Application)

## 7.1 Command Handler

```
@Service
public class DepositarHandler {

    private final EventStore store;
    private final Clock clock;
    private final IdempotencyService idempotency;

    @Transactional
    public void handle(DepositarValor cmd) {
        if (idempotency.jaProcessado(cmd.idempotencyKey())) return;

        var stream = store.load("conta-" + cmd.contaId());     // snapshot +
eventos
        Conta conta = Conta.reconstruir(stream.events());

        conta.depositar(cmd.valor(), cmd.origem(), clock);

        store.append(stream.id(), stream.version(), conta.eventosPendentes(),
                     EventMetadata.of(cmd.correlationId(), cmd.userId()));

        idempotency.registrar(cmd.idempotencyKey());
    }
}
```

## 7.2 Estratégia de retry em conflito

Em `ConcurrencyException`, o handler **recarrega e reexecuta** o comando (até 3 tentativas, com *jitter* curto). Se persistir, a API devolve `409 Conflict`. Como a decisão é refeita sobre o estado novo, a regra de saldo é sempre reavaliada.

## 7.3 Snapshots

- Gerar snapshot a cada **N = 100** eventos (ajustar com métricas).
- Snapshot é **otimização descartável**: o sistema deve funcionar corretamente sem ele.
- Grave a versão do esquema do snapshot; se mudar a classe, descarte e recrie a partir dos eventos.

- Gere o snapshot de forma assíncrona para não aumentar a latência do comando.

### 7.4 Transferência entre contas (Saga)

Como um comando altera apenas um agregado, a transferência é uma **Saga** (process manager), não uma transação única:

```
TransferenciaSolicitada
   → [Saga] comando SacarParaTransferencia (conta origem)
        ├─ ValorSacado → [Saga] comando DepositarTransferencia (conta
destino)
        |      ├─ ValorDepositado → TransferenciaConcluida
        |      └─ falha (conta encerrada/bloqueada) → [Compensação] Estornar
na origem
        |                                                  →
TransferenciaEstornada
        └─ SaldoInsuficiente → TransferenciaRejeitada
```

- O estado da saga também pode ser event sourced ( `transfer-{uuid}` ).
- Cada passo é idempotente e carrega `transferId` como `correlationId`.
- Compensações são novos eventos ( `ValorEstornado` ), nunca remoção de eventos.
- Prazo máximo (*timeout*) por etapa, com tratamento de saga travada.

---

# 8. Lado de Leitura (Query) e Projeções

## 8.1 Read models

```
CREATE TABLE rm_conta_saldo (
    conta_id     UUID PRIMARY KEY,
    saldo        NUMERIC(18,2) NOT NULL,
    limite       NUMERIC(18,2) NOT NULL,
    status       VARCHAR(20)  NOT NULL,
    versao       INT NOT NULL,               -- última versão do stream
aplicada
    atualizado_em TIMESTAMPTZ NOT NULL
);

CREATE TABLE rm_extrato (
    id           BIGSERIAL PRIMARY KEY,
    conta_id     UUID NOT NULL,
    event_id     UUID NOT NULL UNIQUE,         -- idempotência da projeção
    tipo         VARCHAR(30) NOT NULL,
    valor        NUMERIC(18,2) NOT NULL,
    saldo_apos   NUMERIC(18,2) NOT NULL,
    ocorrido_em  TIMESTAMPTZ NOT NULL
);
CREATE INDEX ix_extrato_conta_data ON rm_extrato (conta_id, ocorrido_em
DESC);
```

## 8.2 Projetor idempotente

```
@Transactional
public void on(StoredEvent stored) {
    if (stored.event() instanceof ValorDepositado e) {
        // 'ON CONFLICT DO NOTHING' pelo event_id garante idempotência
        int inserted = jdbc.update("""
            INSERT INTO rm_extrato (conta_id, event_id, tipo, valor,
saldo_apos, ocorrido_em)
            VALUES (?, ?, 'DEPOSITO', ?, ?, ?) ON CONFLICT (event_id) DO
NOTHING
            """, ...);
        if (inserted > 0) {
            jdbc.update("UPDATE rm_conta_saldo SET saldo = saldo + ?, versao
= ? WHERE conta_id = ?", ...);
        }
    }
    checkpoints.advance("extrato", stored.globalPosition());   // mesma
transação
}
```

**Regras de projeção:**

- Projetor e checkpoint são atualizados **na mesma transação**.
- Projetores são idempotentes (reprocessar o mesmo evento não altera o resultado).
- Projetores **não** chamam comandos nem geram eventos de negócio.
- Reconstrução: truncar o read model, zerar o checkpoint e reprocessar tudo (preferencialmente em *blue/green*: construir tabela nova e trocar).

## 8.3 Consistência eventual

| Técnica | Uso |
|---|---|
| Retornar a `versao` no response do comando | O cliente sabe até qual versão precisa ver. |
| Query com `?minVersion=N` | A API aguarda (curto *timeout*) a projeção alcançar a versão. |
| UI otimista | A tela exibe o resultado esperado e confirma depois. |
| **Nunca** decidir regra de negócio lendo o read model | Decisões usam sempre o agregado (fonte da verdade). |

---

# 9. Publicação de Eventos (integração)

O Event Store já é durável, então ele funciona como o próprio *outbox*: um **Event Publisher** lê eventos a partir de um checkpoint e publica no Kafka (at-least-once).

- Chave da mensagem = `streamId` (preserva ordem por conta na partição).
- `eventId` como `messageId`; consumidores deduplicam (Inbox).
- Publique **eventos de integração** (contrato estável e enxuto), não exponha diretamente o formato interno dos eventos de domínio para outros contextos.
- Esquemas versionados em Schema Registry (Avro/JSON Schema) com compatibilidade retroativa.

---

# 10. Evolução de Eventos (Upcasting)

Eventos gravados jamais são editados. Quando o formato muda:

| Mudança | Estratégia |
|---|---|
| Novo campo opcional | Compatível: valor padrão ao ler eventos antigos. |
| Renomear/reestruturar campo | **Upcaster** converte `schema_version 1 → 2` na leitura. |
| Mudar significado do evento | Criar **novo tipo** de evento; manter o antigo suportado. |
| Dividir evento em dois | Upcaster produz dois eventos na leitura. |

```
public class ValorDepositadoV1ToV2 implements Upcaster {
    public boolean supports(String type, int schemaVersion) {
        return type.equals("ValorDepositado") && schemaVersion == 1;
    }
    public JsonNode upcast(JsonNode v1) {
        ObjectNode v2 = v1.deepCopy();
        v2.put("origem", "DESCONHECIDA");   // campo novo em V2
        return v2;
    }
}
```

**Regras:** nunca remover campos usados por versões antigas; manter testes com um *corpus* de eventos reais de cada versão; versionar sempre ( `schema_version` ).

---

# 11. Segurança, LGPD e Auditoria

## 11.1 Dados pessoais em log imutável

O direito de exclusão (LGPD/GDPR) conflita com um log imutável. Estratégias, em ordem de preferência:

1. **Não colocar dados pessoais nos eventos.** Use referências ( `titularRef` ) e mantenha o cadastro em tabela mutável separada.
2. **Crypto-shredding:** quando for inevitável, criptografe os campos pessoais com uma chave exclusiva por titular (AES-GCM, chaves em KMS/Vault). Para "apagar", descarte a chave; os eventos permanecem, mas ilegíveis.
3. Registros que a lei obriga a manter (ex.: transações financeiras) têm base legal de retenção; documente a decisão com o jurídico/DPO.

## 11.2 Demais controles

- TLS no banco, broker e APIs; segredos em cofre.
- Usuário da aplicação com apenas `INSERT` / `SELECT` em `events` .
- Metadados de auditoria ( `userId` , `correlationId` , IP/origem) em todo evento.
- Cadeia de integridade opcional: cada evento guarda o hash do anterior (*hash chain*) para detectar adulteração.
- Backups cifrados e teste periódico de restauração.

---

# 12. Critérios de Aceite

```
CA-01 Depósito
Dado uma conta ativa com saldo 100,00
Quando eu deposito 50,00
Então é gravado o evento ValorDepositado com versão N+1
E o saldo reconstruído pelo agregado é 150,00
E o read model rm_conta_saldo converge para 150,00 em até 1 s

CA-02 Saque sem saldo
Dado uma conta com saldo 100,00 e limite 0,00
Quando eu tento sacar 150,00
Então o comando é rejeitado com SaldoInsuficiente
E nenhum evento é gravado

CA-03 Imutabilidade
Quando qualquer UPDATE ou DELETE é tentado em events
Então o banco rejeita a operação
```

```
CA-04 Concorrência
Dado uma conta com saldo 100,00 e sem limite
Quando 20 saques de 100,00 são enviados simultaneamente
Então exatamente 1 é aceito
E o saldo final é 0,00
E existe exatamente 1 evento ValorSacado

CA-05 Idempotência
Dado um depósito processado com a Idempotency-Key K
Quando o mesmo comando é reenviado com a chave K
Então nenhum novo evento é gravado
E a resposta é equivalente à primeira

CA-06 Reconstrução determinística
Dado qualquer stream
Quando reconstruo o agregado duas vezes a partir dos mesmos eventos
Então o estado é idêntico
E com ou sem snapshot o resultado é o mesmo

CA-07 Time travel
Dado uma conta com eventos ao longo de 30 dias
Quando consulto o saldo na data D
Então recebo a soma apenas dos eventos com occurredAt <= D

CA-08 Reconstrução de projeção
Quando eu apago rm_extrato e reprocesso desde a posição 0
Então o conteúdo final é idêntico ao anterior

CA-09 Projetor idempotente
Quando o mesmo evento é entregue duas vezes ao projetor
Então o extrato não tem linha duplicada e o saldo não é somado duas vezes

CA-10 Transferência com sucesso
Dado contas A (200,00) e B (0,00)
Quando transfiro 80,00 de A para B
Então A = 120,00 e B = 80,00 ao final da saga
E o total do sistema permanece constante

CA-11 Transferência com compensação
Dado que a conta destino está encerrada
Quando transfiro 80,00 de A para ela
Então a saga estorna 80,00 em A com um evento compensatório
E o saldo final de A é o original
E o histórico mantém saque e estorno visíveis
```

```
CA-12 Upcasting
Dado eventos gravados em schema_version 1
Quando o sistema lê esses eventos com o código atual
Então eles são convertidos para a versão atual sem alterar o banco

CA-13 Crypto-shredding
Dado um titular com dados pessoais cifrados nos eventos
Quando a chave desse titular é descartada
Então os dados pessoais deixam de ser recuperáveis
E a integridade financeira do stream permanece

CA-14 Ordem das projeções
Quando 1.000 eventos são gerados concorrentemente
Então nenhum evento é pulado pelo projetor (verificado por contagem e
checksum)
```

---

# 13. Boas Práticas e Anti-padrões

## 13.1 Modelagem de eventos

- Nomeie no passado e no vocabulário do negócio: `ValorDepositado`, não `ContaAtualizada`.
- Evite eventos "CRUD" genéricos (`ContaAlterada`); o motivo da mudança faz parte do valor do padrão.
- Eventos pequenos e focados; inclua dados suficientes para as projeções sem consultar outros sistemas.
- Faça o *Event Storming* com o negócio antes de codificar para descobrir eventos e agregados.

## 13.2 Agregados

- Mantenha-os **pequenos**. Um agregado por fronteira de consistência real.
- Streams com milhões de eventos são sinal de agregado mal delimitado. Exemplo: para contas muito movimentadas, considere fechar o período (evento `PeriodoFechado` com saldo inicial) e iniciar um novo stream.
- Referência entre agregados apenas por id.

## 13.3 Leitura e escrita

- Nunca consulte read models dentro do command handler para decidir regra.

- Cada projeção atende uma necessidade de consulta; crie mais projeções em vez de uma "universal".
- Projete as APIs de comando como *task-based* ( `/depositos` , `/saques` ), não `PUT /contas/{id}` .

## 13.4 Operação

| Métrica | Alerta sugerido |
|---|---|
| `projection_lag_seconds` por projeção | > 5 s por 2 min |
| `eventstore_append_conflicts_total` | Pico incomum |
| `command_duration` p95 | > 100 ms |
| `stream_length` (máx.) | > 5.000 sem snapshot |
| `publisher_lag` | > 10 s |
| `dead_letter_count` | > 0 |

- Propague `correlationId` do HTTP até o consumidor (OpenTelemetry).
- Ferramentas de suporte: consulta de stream por conta, *replay* controlado e reconstrução de projeção.

## 13.5 Anti-padrões

- Usar o Event Store como fila genérica ou como banco para consultas.
- Editar ou apagar eventos "só dessa vez" (corrija com evento compensatório).
- Lógica de negócio, `now()` ou chamadas externas dentro do `aplicar` .
- Compartilhar eventos de domínio internos como contrato público entre serviços.
- Um único projetor monolítico que não pode ser reconstruído.
- Ignorar versionamento de eventos até o primeiro problema em produção.
- Adotar Event Sourcing no sistema inteiro por padrão.

---

# 14. Alternativas de Infraestrutura

| Opção | Prós | Contras | Quando escolher |
|---|---|---|---|
| **PostgreSQL próprio** (este guia) | Controle total, simples de operar, ótimo para aprender. | Você implementa snapshots, subscriptions e ordem global. | Projeto didático, times com forte base em Postgres. |
| **Axon Framework + Axon Server** | Agregados, sagas, projeções e upcasting prontos, integração com Spring. | Acoplamento ao framework; Axon Server tem licença em recursos enterprise. | Entrega rápida em Java/Spring. |
| **EventStoreDB (KurrentDB)** | Feito para ES, subscriptions e projeções nativas, ordem global garantida. | Mais um banco para operar. | ES como estratégia central. |
| **Kafka como log** | Escala enorme. | Não oferece concorrência otimista por stream nem consulta por agregado de forma nativa. | Distribuição de eventos, não como Event Store principal. |

---

# 15. Estratégia de Testes

| Nível | Foco | Ferramentas |
|---|---|---|
| **Domínio (Given-When-Then)** | Dados eventos passados, quando comando, então eventos esperados ou exceção. Sem banco, extremamente rápido. | JUnit 5, AssertJ |
| **Property-based** | Invariantes: saldo nunca abaixo de `-limite`; soma de eventos = saldo; reconstrução determinística. | jqwik |
| **Integração** | Event Store real, concorrência (20 threads), snapshots, projetores. | Testcontainers (PostgreSQL, Kafka) |

| Nível | Foco | Ferramentas |
|---|---|---|
| **Contrato** | Esquemas de eventos de integração. | Schema Registry, Pact |
| **Upcasting** | Corpus de eventos históricos de cada versão. | JUnit + arquivos JSON |
| **Caos/Resiliência** | Derrubar projetor, broker e banco; verificar convergência. | Toxiproxy |
| **Carga** | Throughput de escrita e lag de projeção. | Gatling, k6 |

Exemplo de teste Given-When-Then:

```
@Test
void saqueAcimaDoSaldoEhRejeitado() {
    var conta = Conta.reconstruir(List.of(
        contaAberta(limite("0")), valorDepositado("100")));

    assertThatThrownBy(() -> conta.sacar(new BigDecimal("150"), clock))
        .isInstanceOf(SaldoInsuficienteException.class);

    assertThat(conta.eventosPendentes()).isEmpty();
}
```

---

# 16. Plano de Implementação

| Fase | Entrega | Critérios |
|---|---|---|
| 1 | Event Storming e modelo de eventos; domínio `Conta` com testes Given-When-Then. | CA-01, CA-02, CA-06 |
| 2 | Event Store em PostgreSQL, append com concorrência otimista, imutabilidade. | CA-03, CA-04 |
| 3 | Command API, handlers, idempotência. | CA-05 |
| 4 | Projeções (saldo, extrato), checkpoints, reconstrução. | CA-08, CA-09, CA-14 |
| 5 | Snapshots e time travel. | CA-07 |
| 6 | Saga de transferência com compensação. | CA-10, CA-11 |

---

| Fase | Entrega | Critérios |
|---|---|---|
| 7 | Publisher Kafka, eventos de integração, Schema Registry. | RF-10 |
| 8 | Upcasting, crypto-shredding, hash chain. | CA-12, CA-13 |
| 9 | Observabilidade, carga, caos e documentação operacional. | RNF-03 a RNF-09 |

---

## 17. Decisões Arquiteturais (ADRs resumidos)

| ADR | Decisão | Motivo |
|---|---|---|
| 001 | ES + CQRS apenas nos agregados bancários (Conta, Transferência). | Onde histórico e auditoria têm valor real. |
| 002 | PostgreSQL como Event Store inicial. | Simplicidade e transações ACID no append. |
| 003 | Concorrência otimista por `(stream_id, version)`. | Sem locks longos, alta concorrência. |
| 004 | Transferência via Saga com compensação. | Um comando = um agregado. |
| 005 | Projeções idempotentes e reconstruíveis. | Segurança operacional e novas visões futuras. |
| 006 | Dados pessoais fora dos eventos ou com crypto-shredding. | LGPD com log imutável. |
| 007 | Eventos de integração separados dos de domínio. | Evolução independente dos contextos. |

---

## 18. Glossário

- **Event Sourcing:** persistir mudanças como sequência imutável de eventos; o estado é derivado por replay.
- **CQRS:** separação entre modelo de escrita (comandos) e modelo de leitura (consultas).
- **Stream:** sequência ordenada de eventos de um agregado.
- **Snapshot:** estado consolidado em uma versão, para evitar replay completo.
- **Projection / Read Model:** visão derivada dos eventos, otimizada para leitura.

- **Upcasting:** conversão de eventos antigos para o formato atual durante a leitura.
- **Saga:** coordenação de transações de longa duração entre agregados com compensações.
- **Crypto-shredding:** tornar dados ilegíveis descartando a chave de criptografia.
- **Consistência eventual:** o read model converge para o estado correto, com pequeno atraso.

---

## 19. Referências


- Martin Fowler, *Event Sourcing* e *CQRS* (martinfowler.com).
- Greg Young, *CQRS Documents* e *Versioning in an Event Sourced System*.
- Vaughn Vernon, *Implementing Domain-Driven Design* (agregados e eventos de domínio).
- Documentação do Axon Framework, EventStoreDB/Kurrent e PostgreSQL (`pg_current_snapshot`, constraints).
- Chris Richardson, *Microservices Patterns* (Sagas e Transactional Outbox).